

Let's make sure we have that sequence straight:

- Thread zelda is waiting for something else.
- Because it cannot go on without the something it's waiting for, zelda calls the method `wait()` on itself: `zelda.wait()`;
- Thread zelda remains in the waiting state until the object it is waiting, say scott, announces it is ready by issuing the `notify()` or `notifyAll()` method.

8.6 Thread synchronization

Until this point, all of our threads have lived alone. However, it's possible to have threads share information. In a class, it is common for many threads to all have access to instance variables that have class scope. However, this usually raises issues that could never arise when threads were not involved. For example, what happens when two threads try to write to or read from an instance variable at the same instant?

Coping with this problem raises the thorny subject of thread synchronization.

```
public synchronized int getBalance()  
{  
    ...  
}
```

The first piece of the puzzle is a thing called a monitor. When we say an object is a monitor, we are merely saying that object has the ability to pay attention to the behavior of threads. We accomplish thread synchronization by making methods synchronized. Further, we say every object that contains synchronized threads is in effect a monitor. So in `sumMaya`, if an object is a monitor, then that object permits only one thread at a time to execute a synchronized method on that object.

This is implemented as follows:

- whenever a synchronized method is invoked, the object is locked.
- If my object has a synchronized method, and something calls that synchronized method, then my object is locked up tight.

In fact, for every Java object that contains a synchronized method, Java creates a lock associated with that object.

```
public class LockBox  
{  
    private int contents;  
    private boolean available = false;  
    public synchronized int get()
```